Tech Price Pipeline

Refurbished Laptop Price Monitoring

Rolando Suarez

Data Engineer

Abstract

This document describes the Data Engineering pipeline developed for automated price monitoring of refurbished laptops across Amazon, eBay, and Newegg platforms. The system implements a complete ETL architecture, from data extraction using advanced web scraping techniques, through data transformation and normalization, to interactive visualization in a dashboard developed with Streamlit. The project includes a main orchestrator (`run_pipeline.py`) that unifies all components into an interactive menu interface.

1 Project Overview

The pipeline extracts real-time information from three major e-commerce platforms, processes the data under quality standards, and presents it in an interactive graphical interface. The system is specifically designed for monitoring refurbished laptops (though the scrapers can extract any product from these sites), extracting key features such as brand, RAM, and storage capacity.

1.1 Key Features

- **Automated extraction** with `undetected-chromedriver` (evades anti-bot blocking)
- **Geographic filtering** (eBay: only US products with USD prices)
- **ETL transformation** with data cleaning, normalization, and deduplication
- **Layered architecture**: Raw (JSON) → Silver (CSV) → Gold (SQLite)
- **Interactive dashboard** with Streamlit for comparative analysis
- **Central orchestrator** with interactive menu to run any component combination

2 Pipeline Architecture

The system is organized into four main layers:

2.1 Layer 1: Extraction (Scrapers)

Three independent scrapers use `undetected-chromedriver` to avoid detection:

- **Amazon**: Configures location to Miami (zip code 33101)
- **eBay**: Filters US products and USD prices using URL parameters
- **Newegg**: Standard extraction with specific selectors

2.2 Layer 2: Raw Storage (JSON)

Each execution generates a JSON file with a unique timestamp in `data/raw_*.json`

2.3 Layer 3: Transformation (ETL)

The `data_cleaning.py` script executes:

1. Price cleaning (removes currency symbols and non-numeric characters)
2. RAM extraction using regex patterns
3. Storage extraction (automatically converts TB to GB)
4. Brand detection (APPLE, DELL, HP, LENOVO, ASUS, ACER, SAMSUNG, MICROSOFT, MSI, GIGABYTE)
5. Deduplication by URL and by title+platform

2.4 Layer 4: Final Storage

- **Silver (CSV)**: Clean data backup in `data/silver/`
- **Gold (SQLite)**: Final database at `data/db/tech_prices_gold.db`

3 Project Structure

```
tech-price-pipeline/

        .gitignore                          # Files excluded from repository
        README.md                           # Main documentation
        requirements.txt                    # Project dependencies
        run_pipeline.py                     # MAIN ORCHESTRATOR (interactive menu)

        scrapers/                           # Extraction module
              amazon_monitor.py              # Amazon scraper (with location change to Miami)
              ebay_monitor.py                # eBay scraper (US + USD filter)
              newegg_monitor.py              # Newegg scraper

        processing/                         # Transformation module
              data_cleaning.py               # Complete ETL: cleaning, normalization, loading

        visualization/                      # Visualization module
              dashboard.py                   # Streamlit interactive dashboard

        data/                              # Data (generated locally, NOT uploaded to GitHub)
            raw_amazon_*.json              # Raw Layer: Amazon raw data
            raw_ebay_*.json                # Raw Layer: eBay raw data
            raw_newegg_*.json              # Raw Layer: Newegg raw data
            silver/                        # Silver Layer: CSV backup
                  silver_tech_prices_*.csv
                  silver_tech_prices_latest.csv
            db/                            # Gold Layer: final database
                tech_prices_gold.db        # SQLite with unified products
```

Listing 1: Complete directory structure

4 Main Orchestrator (run_pipeline.py)

The `run_pipeline.py` file is the unified entry point of the system. It implements a `PipelineOrchestrator` class that organizes and executes all pipeline components.

4.1 Orchestrator Functionalities

- **Interactive menu**: Allows selecting which components to execute
- **Sequential execution**: Ensures correct execution order
- **Error handling**: Captures and reports failures in each component
- **Dependency validation**: Verifies required files exist
- **Final summary**: Shows the status of each executed component

4.2 Orchestrator Methods

Method	Description
`run_amazon()`	Executes Amazon scraper
`run_ebay()`	Executes eBay scraper
`run_newegg()`	Executes Newegg scraper
`run_etl()`	Executes ETL processing (`data_cleaning.py`)
`run_dashboard()`	Launches Streamlit dashboard
`run_full_pipeline()`	Executes all components in sequence

Table 1: Main orchestrator methods

4.3 Menu Options

```
=== Tech Price Pipeline - Orquestador ===
1. Ejecutar pipeline completo (Amazon + eBay + Newegg + ETL + Dashboard)
2. Ejecutar solo Amazon + ETL + Dashboard
3. Ejecutar solo eBay + ETL + Dashboard
4. Ejecutar solo Newegg + ETL + Dashboard
5. Ejecutar solo ETL (procesar JSON existentes)
6. Ejecutar solo Dashboard
7. Ejecutar solo Amazon
8. Ejecutar solo eBay
9. Ejecutar solo Newegg
10. Salir
```

Listing 2: Orchestrator interactive menu

5 Technologies Used

Category	Technologies
Language	Python 3.12+
Web Scraping	Selenium, Undetected-Chromedriver
Data Processing	Pandas, Numpy, Regex
Database	SQLite3
Visualization	Streamlit
Operating System	Linux Mint (compatible with Mac/Windows)

Table 2: Technologies used in the pipeline

6 Step-by-Step Installation

6.1 Prerequisites

- Python 3.12 or higher
- Google Chrome or Firefox installed
- Git (optional, for cloning)

6.2 Installation Steps

```
# Clone the repository
git clone https://github.com/RolandoSu03/tech-price-pipeline.git
cd tech-price-pipeline

# Create and activate virtual environment (Linux/Mac)
python3 -m venv venv
source venv/bin/activate

# Install dependencies
pip install -r requirements.txt
```

Listing 3: Clone and configure environment

6.3 Verify Browser Installation

```
# For Chrome
google-chrome --version

# For Firefox
firefox --version
```

Listing 4: Verify browser

7 Pipeline Usage

7.1 Run the Orchestrator (Recommended)

```
# Activate virtual environment
source venv/bin/activate

# Run orchestrator
python run_pipeline.py

# Select an option from the menu
```

Listing 5: Run pipeline with interactive menu

7.2 Run Individual Components

```
# Run only one scraper
python scrapers/amazon_monitor.py
python scrapers/ebay_monitor.py
python scrapers/newegg_monitor.py

# Run only ETL processing
```

```
python processing/data_cleaning.py

# Run only the dashboard
streamlit run visualization/dashboard.py
```

Listing 6: Direct component execution

8 Data Format

8.1 Raw Layer (JSON)

```
{
    "title": "Dell Latitude 7490 16GB RAM 512GB SSD",
    "price_raw": "$329.99",
    "url": "https://www.ebay.com/itm/123456789",
    "platform": "ebay",
    "scraped_at": "2025-05-10 15:30:22"
}
```

Listing 7: Example raw JSON

8.2 Silver Layer (CSV) - Backup

The `data/silver/` folder contains CSV files with already cleaned and normalized data. This layer serves as:

- **Intermediate backup**: If the database fails, structured data is available
- **Rapid verification**: Can be opened with Excel or any text editor
- **Compatibility**: CSV files can be used in other tools without SQL

8.3 Gold Layer (SQLite)

Column	Type	Description
id	INTEGER	Auto-incrementing primary key
title	TEXT	Original product title
brand	TEXT	Detected brand (or "OTHER")
ram_gb	INTEGER	RAM in GB (e.g., 16, 32, 64)
storage_gb	INTEGER	Storage in GB (e.g., 256, 512, 1024)
price	REAL	Clean numeric price
url	TEXT	Unique product URL
platform	TEXT	Platform (amazon, ebay, newegg)
scraped_at	DATETIME	Extraction date and time
last_updated	DATETIME	Last update date

Table 3: Structure of the `products` table in SQLite

9 Geographic Filtering by Platform

Platform	US Filter	Method
eBay	YES	URL parameters (`LH_PrefLoc=1`, `_salic=1`) + currency validation
Amazon	PARTIAL	Location change to Miami (zip code 33101)
Newegg	NO	No specific filters

Table 4: Geographic filtering by platform

10 Common Troubleshooting

10.1 Error 403 on eBay (blocking)

```
# In ebay_monitor.py, increase delays
time.sleep(random.uniform(8, 12))  # Approximately line 45
```

Listing 8: Increase wait times

10.2 RAM or Storage not detected

The regex patterns search for standard formats like "16GB RAM" or "512GB SSD". If products use different formats, modify the patterns in `data_cleaning.py`:

```
patterns = [
    r'(\d+)\s*GB\s*RAM',      # "16GB RAM"
    r'RAM\s*(\d+)\s*GB',      # "RAM 16GB"
    r'(\d+)\s*GB\s*SSD',      # "256GB SSD"
]
```

Listing 9: Extraction patterns

10.3 Chrome/Firefox not found

```
wget -q -O - https://dl.google.com/linux/linux_signing_key.pub | sudo apt-key add
    -
sudo sh -c 'echo "deb [arch=amd64] http://dl.google.com/linux/chrome/deb/ stable
   main" >> /etc/apt/sources.list.d/google-chrome.list'
sudo apt update && sudo apt install google-chrome-stable
```

Listing 10: Install Chrome on Linux

10.4 Dashboard shows no data

```
# Verify database exists
ls -la data/db/tech_prices_gold.db

# Verify it contains data
sqlite3 data/db/tech_prices_gold.db "SELECT COUNT(*) FROM products;"
```

Listing 11: Verify database

11 Suggested Improvements

- **Airflow Orchestration**: Schedule automatic daily executions
- **Email alerts**: Notify when a product price drops
- **Cloud deployment**: Render or Streamlit Cloud for dashboard hosting
- **Structured logging**: Replace `print()` with leveled logs (INFO, ERROR)
- **More platform support**: Best Buy, B&H, Walmart
- **Price history**: Track price changes over time
- **US filter for Amazon and Newegg**: Add additional URL parameters

Credits

Rolando Suarez
Data Engineer

- GitHub: `https://github.com/RolandoSu03`
- Project repository: `https://github.com/RolandoSu03/tech-price-pipeline`

Note: This project respects the terms of service of the web platforms and is exclusively for educational and portfolio purposes.